面向链上链下协同的状态约束数字身份验证方法

马治国[1]，陆昱璁[2]，忻众[2]，何柯阳[1]，李昱锋[3]，刘宇[4]，王者[4]，韩蒙[1*]

（1. 浙江大学，杭州 310000；2. 浙江工商大学，杭州 310000；
3. 华东师范大学，上海 200062；4. 上海交通大学，上海 200240）

*通讯作者：韩蒙（mhan@zju.edu.cn）

摘　要　数字凭据在跨机构业务中用于证明主体资格和授权信息。密钥轮换、凭据撤销及使用场景变化会影响其有效性，仅验证签名难以判断凭据是否满足本次使用条件。针对上述问题，提出一种面向链上链下协同的状态约束数字身份验证方法。该方法通过共享账本维护身份公钥、签发授权和凭据状态，由链下验证服务结合签名与挑战应答，联合核验签发资格、主体控制及凭据有效性。为限制跨服务转用和请求重放，将持有者、完整凭据摘要、接收方及账本上下文绑定至挑战，并持久化保存一次性挑战的消费记录，以拒绝重复提交。针对密钥轮换和凭据撤销，固定单次验证的状态查询高度，依据所观察的账本状态进行判定。原型由身份状态合约和链下验证服务组成，在单机双以太坊节点、双验证服务环境下完成9轮实验，共产生3752条记录，各类判定均符合用例预期。独立性能实验中，150条正常验证请求的服务端处理时间中位数和第95百分位数分别为54.459 ms和94.976 ms。实验结果表明，该方法能够依据所观察的链上状态识别失效凭据和密钥，拒绝主体或接收方错配的请求，以及使用已消费挑战的重复提交，实现身份状态管理与链下凭据验证的协同。

关键词　数字身份；可验证凭据；状态约束；挑战应答

State-Constrained Digital Identity Verification for On-Chain and Off-Chain Collaboration

Abstract　Digital credentials are used to establish subject qualifications and authorization in cross-organizational applications. However, key rotation, credential revocation, and changes in usage contexts affect credential validity, making signature verification alone insufficient to determine whether a credential satisfies the conditions of a particular use. To address these issues, a state-constrained digital identity verification method for on-chain and off-chain collaboration is proposed. The method maintains identity public keys, issuer authorization, and credential status on a shared ledger. Off-chain verification services combine signature verification with challenge-response procedures to jointly verify issuer authorization, holder control, and credential validity. To restrict cross-service reuse and request replay, challenges bind the holder, the digest of the complete credential, the intended verifier, and the ledger context. Persistent records of one-time challenge consumption are used to reject repeated submissions. To account for key rotation and credential revocation, state queries within each verification are performed at a fixed block height, and decisions are made according to the observed ledger state. A prototype comprising an identity-state contract and off-chain verification services is evaluated through nine experimental runs with two Ethereum nodes and two verification services on a single host. The experiments produce 3,752 records, with all outcomes matching the corresponding test expectations. In an independent performance experiment involving 150 normal verification requests, the median and 95th-percentile server-side processing times are 54.459 ms and 94.976 ms, respectively. The results show that the method identifies invalid credentials and keys according to the observed on-chain state and rejects holder mismatches, intended-verifier mismatches, and repeated submissions of consumed challenges, enabling coordinated identity-state management and off-chain credential verification.

Key words　digital identity; verifiable credential; state constraint; challenge-response

1　引言

数字内容流通、跨机构数据交换和数字贸易需要在不同业务系统之间确认主体身份与使用权限。数字凭据将主体资格和授权信息交由签发者声明，使接收服务能够据此处理访问与交易请求。W3C可验证凭据模型区分签发者、持有者和验证者，为凭据的签发、呈示与验证提供了统一的角色划分[1]。区块链则通过共享登记记录，为跨服务查询身份公钥、签发权限和凭据状态提供依据。

围绕公钥可信管理与凭据使用，已有研究形成了不同的技术途径。Certcoin研究去中心化环境中的公钥基础设施及身份保留[2]，将公钥管理纳入共享记录。OpenID凭据呈示协议通过随机数及验证者绑定限定呈示上下文[3]。这些工作分别从公钥管理和请求绑定等方面支持身份验证。在实际业务中，验证服务还需要将这些检查与当前授权状态结合，判断所呈示的凭据是否适用于本次请求。

凭据的签名在生成后保持不变，签发资格、主体密钥和撤销状态则可能随业务操作发生变化。因此，签名核验需要结合凭据使用时的状态，才能判断签发者是否仍获授权、持有者是否控制当前密钥，以及凭据是否已被撤销。链上状态更新与链下验证分别执行时，同一次验证中的多项查询还需要采用明确的观察时点，避免将不同状态下的信息用于同一次判定。

凭据使用也受到接收方和业务上下文的约束。主体、完整凭据、接收服务及账本环境需要与本次挑战共同绑定，已完成的挑战则需要保留消费记录，以识别跨服务转用和历史请求重放。对于数字内容业务，TrustGen通过视觉指纹与链上登记关联内容对象及声明主体[4]。在这一关联基础上，业务服务还需检查当前请求者是否控制登记密钥、所用凭据是否有效，才能依据验证结果执行后续授权。

针对上述问题，本文提出一种面向链上链下协同的状态约束数字身份验证方法。该方法由共享账本维护身份公钥、签发授权和凭据状态，链下验证服务结合签名核验与一次性挑战，对主体控制、凭据有效性及使用上下文进行联合判定。单次验证固定状态查询高度，并记录请求摘要、判定原因和关联区块，使验证结果能够对应到明确的账本状态。现实身份与资格的核验由登记治理及签发流程承担。

本文的主要工作与贡献如下：

（1）设计状态约束的凭据验证方法，将签发资格、主体与当前密钥的对应关系、有效时间窗及撤销状态纳入同一接受条件。通过固定单次查询高度，使链下验证依据一致的账本视图处理密钥轮换、凭据撤销和身份停用。

（2）设计绑定完整凭据摘要、持有者、接收方及账本上下文的挑战应答流程，分别使用签名域区分凭据、挑战和呈示，并持久化保存一次性挑战的消费记录，以约束跨服务使用和重复提交。

（3）实现身份状态合约与独立验证服务，在单机双节点账本、双验证服务环境下开展功能及性能实验。通过撤销、轮换、请求重放、接收方错配和状态查询故障等用例核对判定结果，并通过独立记录复核与性能测量分析状态读取和密码验证的开销。实验结果表明，该方法能够依据所观察的链上状态识别失效凭据和密钥，并拒绝上下文不符及使用已消费挑战的请求。

本文其余部分安排如下：第2节梳理身份标识、凭据状态管理及内容关联的相关研究，明确本文的验证对象。第3、4节分别给出系统模型与验证目标，以及结合链上状态和一次性挑战的凭据验证流程。第5节分析主体控制、状态约束、重放防护及验证复杂度。第6、7节介绍原型实现与实验设计，分析功能正确性、验证处理性能及服务接入条件。第8节总结状态约束验证方法的设计及实验所得结论。

2 相关工作

2.1 标识、凭据和业务真实性

去中心化标识符（Decentralized Identifier，DID）核心规范区分标识主体、控制者及验证关系[5]，可验证凭据模型区分签发者、持有者和验证者[1]。本方法据此组织签发、呈示及验证，以签发资格、主体绑定、有效期和撤销状态约束接受判定。身份验证需要明确公钥的信任来源及其当前用途。RFC 5280 以信任锚为起点检查证书链、有效期和用途[6]。Certcoin 研究去中心化环境中的公钥基础设施及身份保留[2]。ERC-1056 通过控制者、委托者和属性操作管理身份状态[7]。本文通过共享账本维护认证公钥及签发资格，在每次呈示中检查密钥版本、身份启用状态与签发授权。身份登记、现实姓名和机构资格的核验由治理与签发流程承担。

原型消息采用受限的规范JSON格式，数据类型限定为ASCII字符串和整数。链上以身份标识的摘要进行索引，呈示中传递完整凭据。DID方法互操作、JSON-LD与选择性披露可作为后续格式扩展。

JWT定义签发者、主体、受众、到期时间及令牌标识等声明[8]。W3C Data Integrity规范规定证明生成、验证及密码套件的数据处理框架[9]。本文采用自定义签名域与受限JSON，将受众、挑战和完整凭据摘要纳入签名输入。

2.2 凭据状态、呈示新鲜性与隐私

W3C Bitstring Status List以紧凑列表表达撤销和挂起状态[10]。OpenID for Verifiable Presentations通过nonce及验证者绑定限定呈示上下文[3]。本文逐一查询凭据的链上状态，将接收方、凭据摘要和账本上下文写入挑战及持有者签名，并在同一接受流程中处理密钥轮换、状态快照与挑战消费记录的持久化。Coconut结合门限签发、重随机化及选择性披露构造隐私凭据[11]。本文按第3节的接受条件验证完整的受限凭据，呈示所涉及的凭据标识、状态查询及请求关系仍可被关联。

2.3 内容关联机制与身份验证的衔接

TrustGen通过视觉指纹与链上登记关联内容对象及声明主体[4]。该方法利用DINOv2提取384维特征[12]，通过主成分分析降维[13]，再经QR矩阵分解重正交化、归一化及整数化得到256维指纹。随后，利用向量检索库FAISS检索候选对象[14]，并通过XFeat执行局部几何匹配[15]。该流

程输出的候选内容及登记主体作为身份组件的业务输入。DECORAIT研究生成内容归属及报酬分配[16]，ARCHANGEL以账本记录数字档案完整性依据[17]。本文在内容与登记记录的关联基础上，检查当前请求者的身份控制及凭据状态。C2PA规范通过签名清单描述内容来源和处理历史[18]，IPFS采用内容寻址方式组织存储对象[19]。业务接入时，根据来源记录与内容标识建立对象映射，并通过挑战应答验证身份。

内容关联与身份验证的接口关系如图1所示。内容关联流程采用TrustGen的视觉指纹检索与几何核验方法[4]，输出候选对象及登记主体，供身份验证流程进一步核验凭据主体、呈示者和账本状态。对于涉及多个贡献者的对象，保留各主体与对象的关联关系，映射、声明绑定和授权规则见第4.4节。表1列出各机制的验证对象，本文的性能实验聚焦其中的身份组件，测量状态查询、签名核验和挑战消费开销。

3 系统模型与验证目标

3.1 参与方和信任边界

系统包括治理方G、签发者I、持有者H、验证者V以及共享账本L。治理方登记具有签发资格的身份，身份控制者通过以太坊账户执行身份登记、密钥轮换及停用。签发者生成凭据并登记摘要后，持有者使用Ed25519认证密钥生成呈示，验证者通过自己的链节点读取状态并执行检查。控制链上交易的账户密钥与链下Ed25519认证密钥分离。

各验证服务分别从各自的链节点读取状态，使用独立的挑战签名密钥和nonce数据库，并通过账本共享身份状态。本方法的去中心化体现在身份验证依据由共享账本维护，身份控制者通过自身密钥完成呈示，各验证服务能够依据账本状态独立核验。签发权限由合约治理账户管理，现实身份与资格由登记和签发流程核验。

攻击模型包含字段及密钥替换、历史呈示重放、未授权签发及越权状态修改。分析假定诚实方私钥安全、Ed25519不可伪造、SHA-256满足所需的抗碰撞性，并信任登记治理、验证服务、时钟、数据库和所读取的账本视图。

3.2 链上状态与协议对象

在区块高度b，身份标识d对应状态

$$S_d^{(b)} = (a_d, pk_d, \alpha_d, \iota_d, v_d) \tag{1}$$

其中a_d为控制者账户，pk_d为认证公钥，α_d为启用状态，ι_d为签发权限，v_d为密钥版本。记H为SHA-256摘要函数，身份映射以$\mathrm{H}(d)$索引，凭据登记状态包含签发者摘要、主体摘要、正文摘要和撤销位。

凭据正文B含格式版本、凭据标识、签发者、主体、生效时间、到期时间及声明，其中B.subject为凭据声明的主体身份标识。令Can表示受限规范序列化，Sign表示签名算法，sk_I为签发者的认证私钥。以D_C, D_Q, D_P分别标记凭据、挑战和呈示的签名域，$\|$表示字节串连接。签发者生成凭据签名σ_I，并与正文组成完整凭据C:

$$\sigma_I = \mathrm{Sign}_{sk_I}(D_C \,\|\, \mathrm{Can}(B)), \quad C = (B, \sigma_I) \tag{2}$$

链上以$\mathrm{H}(\mathrm{Can}(B))$核对凭据正文，挑战与呈示以完整凭据$C$的摘要同时绑定正文和签发者签名。

设挑战随机量长度为λ_r位。验证者均匀生成nonce r，挑战正文为

$$Q = (r, H, \mathrm{H}(\mathrm{Can}(C)), V, \Gamma, t_0, t_1) \tag{3}$$

其中H和V分别为持有者与接收方的身份标识，Γ包含链标识、创世区块哈希及身份合约地址，t_0, t_1分别为挑战生效与到期时刻，$[t_0, t_1)$为挑战可用窗口。验证者签署$D_Q \,\|\, \mathrm{Can}(Q)$，持有者将挑战、完整凭据摘要、自己的标识及公钥放入呈示正文P，再签署$D_P \,\|\, \mathrm{Can}(P)$。其中，$P$.holder为呈示中声明的持有者身份标识。

三类签名域采用固定编码，并与相应消息的规范序列化结果连接。签名采用RFC 8032定义的Ed25519[20]。具体域字符串与参数取值见第6节。

3.3 接受谓词

令t_v为验证函数开始处理请求时读取的服务端时间，b为本次链状态查询固定的区块高度。令A_{fmt}表示结构及规范编码符合要求，A_{ctx}表示接收方和链上下文一致，A_{time}表示凭据及挑战在t_v处于各自的有效时间窗。A_{once}表示本次请求已完成挑战的一次性领取并持久化消费状态，A_{state}表示高度b的登记状态允许使用，A_{bind}表示主体、持有者及该状态中的当前密钥之间的绑定关系成立，A_{sig}表示三类签名均验证通过。记上述七项检查的下标集合为$\mathcal{K}$。验证者仅在所有检查均通过时接受请求，即

$$\mathrm{Accept}(C, P; b, t_v) = \bigwedge_{k \in \mathcal{K}} A_k \tag{4}$$

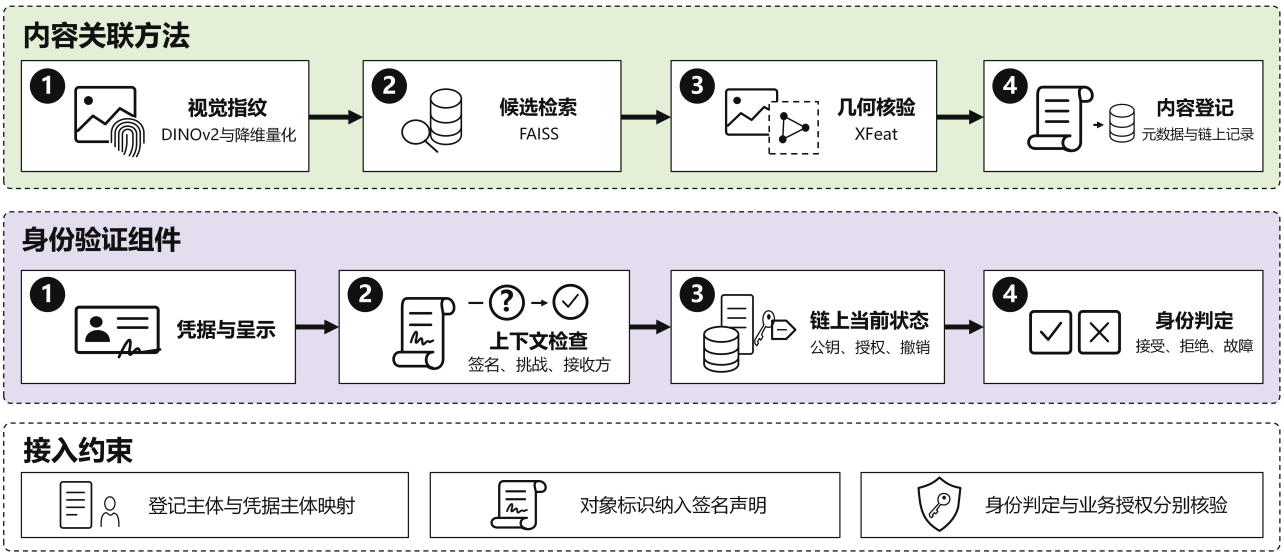


图 1 内容关联服务与身份验证组件的接口关系

表 1 身份协作机制的验证对象与关联条件

机制	验证对象	关联条件	本文中的位置
视觉指纹匹配	相近内容及候选来源	内容标识与登记主体映射	内容关联业务输入
凭据验签	凭据字节与签名密钥	可信公钥、签发资格及有效期	接受谓词中的签名条件
身份及凭据登记	公钥、签发权限及状态	当前控制密钥与链下请求绑定	状态查询依据
本文验证组件	主体控制、签发资格、凭据状态及一次性呈示	可信账本视图、密钥保护和签发治理	身份有效性判定

A_{state}检查签发者启用且获授权、持有者启用、凭据已登记且处于有效状态，以及登记正文摘要和双方标识一致。A_{bind}要求$B.\text{subject} = P.\text{holder}$，呈示公钥等于持有者在高度$b$的当前公钥。

4 状态约束验证方法

4.1 登记、签发和呈示流程

图2包含管理与使用两条路径。管理路径写入身份及凭据状态，使用路径在链下处理凭据正文、签名和挑战，并从账本读取验证所需状态。Registry.register用于登记控制者账户、初始公钥、启用状态和版本，治理账户通过authorizeIssuer管理签发权限。签发登记时，调用者须为已启用且获授权的签发者的控制者，同时要求主体身份处于启用状态、凭据标识可用。账户权限检查和状态转换由以太坊合约执行[21]，Ed25519正文签名由链下服务验证。验证者生成并保存挑战正文及使用状态，持有者将其绑定到完整凭据。接收方与账本三元组同时进入签名范围。挑战有效期记为$\Delta_{\text{ch}} = t_1 - t_0$，判定时间取服务端开始处理请求的时刻。

4.2 分阶段验证与一次性消费

数字签名要求明确消息的字节表示。RFC 8785给出了JSON规范化方案[22]。本原型采用受限序列化，将数据类型限定为ASCII字符串和整数，并设置相应的格式错误用例以检查拒绝行为。验证过程按以下顺序实施。

1. 解析请求，拒绝缺失字段、重复JSON键、非规范Base64及不符合格式的密钥与时间字段。受限规范化不接受浮点数，避免多个字节表示被误认为同一签名对象。
2. 检查接收方和链上下文。随后验证挑战签名，核对挑战中的持有者、完整凭据摘要及本地保存的挑战正文。
3. 检查本地挑战尚未使用，使用SQLite原子条件更新将nonce置为“已使用”，并提交数据库事务。挑战已被消费或条件更新未成功时拒绝请求。提交成功后，再检查挑战时间窗。
4. 读取固定区块高度的账本状态，依次核验签发权限、凭据时间窗、签发者签名、主体绑定、持有者启用状态及当前公钥，再核验持有者签

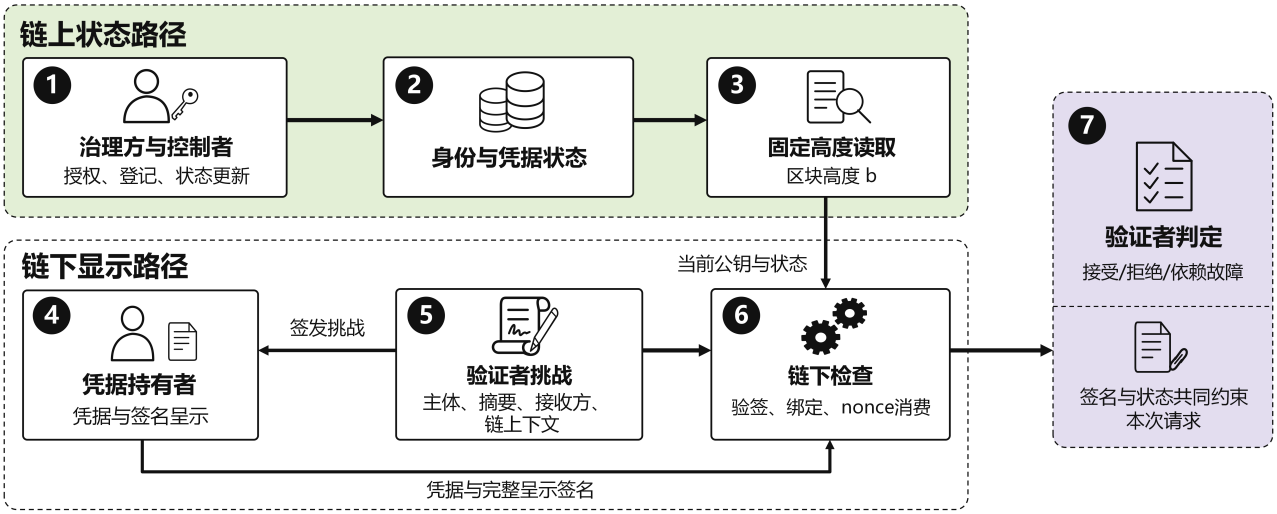


图 2 身份状态管理与链下呈示验证。箭头表示管理路径和验证依赖。验证者签发挑战，持有者签署呈示，验证者结合账本状态作出判定。

名和凭据登记、撤销状态。全部检查通过后接受该次呈示。

5. 保存判定原因、可用的关联区块、请求摘要及处理时间。远程过程调用（RPC）读取失败时，返回依赖故障状态，已提交的挑战消费记录保持有效。

原型将每个挑战用于一次验证尝试。步骤3完成消费后，即使后续检查失败或状态读取发生故障，消费记录仍予以保留，客户端需重新获取挑战后发起下一次验证。若请求在步骤1、2被拒绝，则保持原挑战状态。各验证服务持久化保存消费记录，以拒绝已使用挑战的再次提交及服务重启后的重放。

4.3 固定高度的状态约束

ERC-1271以isValidSignature支持依赖合约状态的账户验签[23]。本文组件采用登记的Ed25519公钥及版本进行链下验签，相关状态固定在同一区块高度读取。验证服务先读取本节点最新区块并获得高度b，再以相同高度执行身份、主体和凭据状态的eth_call。这样可避免多次读操作分别落在不同高度，形成“旧身份密钥与新凭据状态”混合的局部视图。结果同时保存该区块的信息，使离线复核能关联到实际观察状态。

当前实现通过区块高度指定状态，并依赖可信节点视图。EIP-1898提供按blockHash查询及requireCanonical选项[24]，后续可据此扩展固定查询区块的方式。时间判定采用请求开始时刻t_v，状态判定采用观察高度b，这两个观测值在本次处理过程中保持不变。凭据随后到期或撤销时，后续请求按新的时刻与状态重新检查。不同节点的观察高度存在差异时，可能暂时作出不同的判定。查询节点观察到撤销交易后，后续请求因撤销位为真而被拒绝。密钥采用严格的当前版本策略：持有者轮换后使用新密钥呈示，签发者轮换后完成凭据重新签发。

4.4 面向内容服务的接入约束

内容接入将候选对象、登记主体和请求关联到凭据声明，并分别执行身份判定与业务授权。具体接口条件如下。首先，将登记主体映射到身份标识d，核对$d = B.\mathrm{subject} = P.\mathrm{holder}$，多贡献者对象逐一保留主体关系。其次，将规范化对象标识、记录版本和用途写入凭据声明，经签发和登记后由挑战绑定完整凭据。最后，业务服务结合式(4)及操作授权策略处理请求。目前已完成的身份组件独立测试覆盖凭据完整性、主体绑定、接收方约束和状态变化，内容服务接入仍处于接口设计阶段。后续集成需验证主体映射、对象替换、撤销状态传递及业务记录与身份判定的关联。

5 安全性与复杂度分析

5.1 主体控制与凭据绑定

命题1. *在信任模型成立、登记状态正确且式(4)中的各项检查均按实现流程执行的条件下，接受的呈示同时绑定至高度b处启用的持有者身份、该状态中的认证公钥、已授权签发者及未撤销的登记凭据。*

证明. 接受要求呈示公钥等于链上当前公钥且持有者签名通过。规范序列化与域分离固定签名对象；签发者签名和登记摘要绑定凭据内容；签发资格、主体启用和撤销状态由接受谓词检查。在既定信任模型下，接受请求满足上述绑定关系。

5.2 状态变化与约束检查

命题2. *设凭据撤销被包含于区块b_r。若验证者读取同一条未发生相关重组的有效账本历史，查询高度$b \geq b_r$，且撤销位此后保持为真，则不论旧签名是否仍然正确，式(4)均不接受该凭据。*

证明. 在给定历史及高度条件下，状态查询返回撤销位为真，故$A_{\mathrm{state}} = 0$，请求被拒绝。$b < b_r$时读取撤销前状态，判定按该状态执行。

公钥在区块高度b_k处轮换后，按$b \geq b_k$读取新状态时，旧持有者密钥不满足A_{bind}。表2列出各项检查对应的请求情形，其中接收方专项实验仅修改接收方字段，用于检验该条件的独立约束作用。

5.3 上下文隔离与重放约束

接收方、链标识、创世区块或合约地址变化时，上下文检查拒绝原挑战。新的上下文需取得相应验证者的挑战签名及持有者的呈示签名。独立均匀抽取λ_r位nonce时，q次挑战的随机碰撞概率至多为$q(q-1)/2^{\lambda_r+1}$。原型使用单服务持久化数据库处理重启后的重复请求；多副本消费需要共享一致性存储或划分挑战域。

5.4 可追踪性、隐私和可用性

判定记录保存请求摘要、观察高度及相关状态，离线复核时依据交易输入、事件和签名重建判断过程。身份登记与签发事实由治理和签发流程负责核验。链上保存身份、公钥及凭据摘要，完整声明发送给验证者。公开的标识、状态变化及访问模式可能被关联，短小声明也可能通过对公开摘要进行字典匹配而被识别。一次验证包含固定数量的签名检查、nonce条件更新及状态读取。消息长度为ℓ时，规范化与摘要处理为$O(\ell)$。登记和轮换属于状态写入交易，其确认时间与验证服务处理时间分别测量，以区分链上确认与链下处理开销。

6 原型实现与实验设计

6.1 环境、数据与复核方法

身份认证公钥采用32 字节Ed25519 编码，挑战随机量长度$\lambda_r = 256$ 位，默认有效期$\Delta_{\mathrm{ch}} = 600$ 秒。凭据、挑战及呈示的签名域分别为I21-VC-v1 、I21-CH-v1 和I21-VP-v1 ，均以零字节结尾。原型由链上身份状态合约和链下验证服务组成。状态合约负责身份登记、签发授权、公钥轮换、凭据登记及撤销管理；验证服务负责挑战生成、凭据与呈示检查、一次性挑战消费及判定输出，状态查询模块读取同一区块高度的身份与凭据信息。编码模块统一消息格式与签名域，验证记录关联请求摘要、处理时刻和所观察区块。实验通过独立重验签名、重建状态及对照预期判定进行交叉验证。软件环境采用Python 3.12.14、Geth 1.13.15和Solidity 0.8.24，合约编译目标为Paris 版以太坊虚拟机。认证实现使用cryptography 50.0.1，独立密码验证使用Node.js 24.19。两个Geth 进程分别配置数据目录、签名密钥和RPC 端口，通过P2P 连接组成测试网络。两个HTTP 验证服务分别使用独立的签名密钥与SQLite 文件。实验系统为Windows 11（10.0.26200），处理器标识Intel64 Family 6 Model 170，14 个逻辑处理器。验证服务采用单线程HTTPServer ，两个链节点和服务位于同一主机，使用合成身份数据。

三个正式正确性批次各登记100个持有者，对28类用例分别执行30次，共2520条记录。每类选取30个目标持有者，主体错配用例另含呈示身份。不同运行使用新链配置和随机测试密钥。除正确性主批次外，还完成了包含28条记录的一轮冒烟测试、包含84条记录的一轮回归测试、一轮包含10条预热和840条正式请求的性能实验，以及三轮各90条记录的接收方隔离实验。上述已完成实验共产生3752条记录。

6.2 负例设计与单条件隔离

负例覆盖消息结构、签名、主体绑定、授权、时间、状态和交易权限。未授权签发者及公钥替换者使用各自有效密钥重新签名。越权修改和撤销用例提交链上交易，并检查回滚回执及状态。前6轮的124条跨验证者挑战同时改变了接收方、签名归属及nonce来源。接收方专项实验由当前验证者签署并保存挑战，仅修改接收方字段，形成90条单条件负例及180条正常对照，结果单独统计。

6.3 计时与判定口径

服务端采用高精度计时，覆盖解析、状态查询、验签及nonce处理，客户端往返、凭据签发和最终日志落盘时间另行统计。RPC累计时间与其余链下处理时间分别记录，后者包含签名、调度及数据库操作，以区分状态访问与本地处理的开

表 2 身份验证条件及其拒绝情形

验证条件	检查对象或请求情形	判定规则
签发资格与主体绑定	攻击者以自己的密钥签发或呈示受害者标识	按正确登记核验签发资格、主体及当前公钥
当前凭据及密钥状态	撤销前凭据或轮换前密钥	按观察高度b查询并拒绝失效状态
接收方及账本上下文	跨服务或跨部署使用呈示	挑战与呈示共同绑定指定上下文
一次性挑战	重复提交此前接受的呈示	持久化条件更新限制重复领取

表 3 主要实验类别与验证结果

实验类别	实际观察	支撑范围
正文和签名篡改	修改正文、凭据签名或持有者签名后拒绝	签名与规范消息绑定
公钥替换或主体错配	攻击者重新签名后仍被拒绝；主体错配被拒绝	当前身份密钥及subject-holder绑定
撤销、轮换和停用	撤销或停用后拒绝；新持有者密钥接受、旧密钥拒绝；签发者轮换后旧凭据拒绝	验证结果随可观察链上状态改变
nonce重放及重启重放	已消费挑战再次提交时拒绝	单服务持久化消费状态
接收方隔离专项	90条单条件负例拒绝，180条正常对照接受	接收方字段有独立约束作用
越权修改与撤销	实际链上交易回滚，状态未发生目标修改	身份控制者权限及撤销权限由合约执行
状态查询不可达	124条预设故障返回依赖错误	查询失败分支

表 4 服务端及签发处理时间
（单位：ms；P95表示第95百分位数）

阶段	数量	中位数	P95
正常验证	150	54.459	94.976
正常验证RPC累计	150	38.376	76.189
正常验证链下差额	150	16.200	20.885
业务拒绝（混合）	600	36.607	93.548
状态不可达处理	30	264.244	272.412
凭据签名与序列化	250	0.137	0.231

销。正常验证性能仅统计独立性能实验中正式接受的150条请求，排除10条预热记录。600条业务拒绝和30条状态不可达记录分别统计，另有60条权限交易用例不并入验证服务的拒绝计时。P95为第95百分位数，取样本按升序排列后的第$\lceil 0.95 n_{\mathrm{obs}} \rceil$项，其中$n_{\mathrm{obs}}$为该组观测样本数。链上管理交易从提交开始计时，至程序观察到回执时结束，其中包含排队、轮询和顺序记录的开销。实验未固定背景负载及操作系统缓存，统计结果仅描述该运行环境下的观测情况。

7 实验结果与讨论

7.1 功能正确性及状态转换

已完成实验的3752 条记录中，810 条接受、2818 条拒绝、124 条返回预设依赖故障，判定均符合相应用例预期，覆盖正常请求与异常处理情形。表3列出主要结果。交叉验证采用独立实现，通过Node/OpenSSL 执行9024 次Ed25519 密码运算，检查2790 笔原始签名交易，并重建2781 次非部署合约操作。此外，还复算了567 个区块头、558 个非创世Clique 签名及1134 个交易树或回执树根，以核对请求、密码运算与链上状态记录的一致性。

7.2 验证处理性能

当前实现逐次查询DID和凭据状态，正常请求的处理时间中位数为54.459 ms，其中RPC累计时间的中位数为38.376 ms，状态访问是主要开销来源。表4中的各项统计值均根据逐次记录计算。业务拒绝集合包括结构检查失败和状态查询后的拒绝。RPC故障以不可达地址触发，其处理时间中位数约264 ms，主要对应超时及异常处理。身份登记、凭据登记和撤销交易从提交到观察到回执的耗时中位数分别为2378.636、3710.547和2174.559 ms。第二节点观察已确认区块的13次样本覆盖多个操作阶段，等待时间中位数为2.951 ms，P95为27.449 ms。

7.3 服务接入与部署配置

内容业务按对象摘要定位声明，调用身份组件验证呈示，再依据业务策略执行授权操作。该接入流程与第4.4节的对象、主体和会话接口一致。跨机构部署时，需配置状态滞后处理策略和挑战消费记录的存储方式，并明确身份恢复的治理机制。内容服务还需按照第4.4节的接口设计，完成主体映射与业务授权的联调。

8 结论

本文提出一种状态约束的数字身份验证方法，将共享账本中的身份公钥、签发授权及凭据状态，与链下签名检查和挑战应答相结合，以处理时刻及观察到的账本状态为依据，对主体控制、有效期、撤销、密钥轮换及使用上下文进行联合判定。原型由身份状态合约与链下HTTP验证服务组成。双节点账本和双验证服务的实验表明，撤销、停用及密钥轮换能够改变后续呈示的接受结果，持久化保存的挑战消费记录能够用于拒绝重复提交。接收方专项实验的90条单条件负例全部被拒绝，验证了受众绑定的独立作用。正常请求的服务端处理时间中位数为54.459 ms，状态访问是主要开销来源。该方法通过联合核验共享账本状态与链下呈示，使凭据仅在主体绑定、使用上下文及所观察到的授权状态均符合要求时被接受，实现链上身份管理与链下验证的协同。

致谢

本研究得到国家重点研发计划项目“基于区块链的新型信任体系”（2023YFB2704400）资助。

参考文献

[1] W3C. Verifiable Credentials Data Model v2.0[S/OL]. 2025-05-15. https://www.w3.org/TR/2025/REC-vc-data-model-2.0-20250515/.
[2] Fromknecht C, Velicanu D, Yakoubov S. A Decentralized Public Key Infrastructure with Identity Retention[R/OL]. IACR Cryptology ePrint Archive, 2014/803. https://eprint.iacr.org/2014/803.
[3] OpenID Foundation. OpenID for Verifiable Presentations 1.0[S/OL]. 2025. https://openid.net/specs/openid-4-verifiable-presentations-1_0.html.
[4] Ma Z, Lu Y, Xin Z, et al. TrustGen: Robust On-Chain Provenance for Generative AI via Orthogonalized Visual Fingerprinting and Lightweight Matching[C]//2026 IEEE International Conference on Software Services Engineering (SSE). IEEE, 2026: 67–77.
[5] W3C. Decentralized Identifiers (DIDs) v1.0[S/OL]. 2022-07-19. https://www.w3.org/TR/2022/REC-did-core-20220719/.
[6] Cooper D, Santesson S, Farrell S, et al. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile[S]. RFC 5280, 2008. https://www.rfc-editor.org/rfc/rfc5280.html.
[7] Braendgaard P, Torstensson J. ERC-1056: Ethereum Lightweight Identity[EB/OL]. (2018-05-03)[2026-09-30]. https://eips.ethereum.org/EIPS/eip-1056.
[8] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT)[S]. RFC 7519, 2015. https://www.rfc-editor.org/rfc/rfc7519.
[9] W3C. Verifiable Credential Data Integrity 1.0[S/OL]. 2025-05-15. https://www.w3.org/TR/2025/REC-vc-data-integrity-20250515/.
[10] W3C. Bitstring Status List v1.0[S/OL]. 2025-05-15. https://www.w3.org/TR/2025/REC-vc-bitstring-status-list-20250515/.
[11] Sonnino A, Al-Bassam M, Bano S, et al. Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers[C]//Proceedings of the 2019 Network and Distributed System Security Symposium. Internet Society, 2019.
[12] Oquab M, Darcet T, Moutakanni T, et al. DINOv2: Learning Robust Visual Features without Supervision[J/OL]. Transactions on Machine Learning Research, 2024. https://openreview.net/forum?id=a68SUt6zFt.
[13] Abdi H, Williams L J. Principal Component Analysis[J]. WIREs Computational Statistics, 2010, 2(4):433–459.
[14] Johnson J, Douze M, Jégou H. Billion-Scale Similarity Search with GPUs[J]. IEEE Transactions on Big Data, 2021, 7(3): 535–547.
[15] Potje G, Cadar F, Araujo A, et al. XFeat: Accelerated Features for Lightweight Image Matching[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. IEEE, 2024: 2682–2691.
[16] Balan K, Black A, Gilbert A, et al. DECORAIT–DECentralized Opt-in/out Registry for AI Training[C]//Proceedings of the 20th ACM SIGGRAPH European Conference on Visual Media Production. New York: ACM, 2023: 1–10.
[17] Collomosse J, Bui T, Brown A, et al. ARCHANGEL: Trusted Archives of Digital Public Documents[C]//Proceedings of the ACM Symposium on Document Engineering 2018. New York: ACM, 2018: 1–4.
[18] Coalition for Content Provenance and Authenticity. C2PA Technical Specification, Version 1.3[S/OL]. 2023-04. https://spec.c2pa.org/specifications/specifications/1.3/specs/C2PA_Specification.html.
[19] Benet J. IPFS—Content Addressed, Versioned, P2P File System[R/OL]. arXiv:1407.3561, 2014. https://arxiv.org/abs/1407.3561.
[20] Josefsson S, Liusvaara I. Edwards-Curve Digital Signature Algorithm (EdDSA): RFC 8032[R/OL]. RFC Editor, 2017. https://www.rfc-editor.org/info/rfc8032/.
[21] Wood G. Ethereum: A Secure Decentralised Generalised Transaction Ledger[R/OL]. Shanghai版本. 2023. https://ethereum.github.io/yellowpaper/paper.pdf.
[22] Rundgren A, Jordan B, Erdtman S. JSON Canonicalization Scheme (JCS)[S]. RFC 8785, 2020. https://www.rfc-editor.org/rfc/rfc8785.
[23] Giordano F, Condon M, Castonguay P, et al. ERC-1271: Standard Signature Validation Method for Contracts[EB/OL]. (2018-07-25)[2026-10-01]. https://eips.ethereum.org/EIPS/eip-1271.
[24] Cooper C. EIP-1898: Add blockHash to defaultBlock methods[EB/OL]. (2019-04-01)[2026-10-01]. https://eips.ethereum.org/EIPS/eip-1898.